# Lecture 23: The Yau–Yau Filtering Method
# Nonlinear Filtering via the Duncan–Mortensen–Zakai Equation

Q. Deng

January 9, 2026

# Roadmap

1. Nonlinear filtering: formulation
2. From DMZ to the Yau–Yau filter (memoryless real-time solution)
3. Recent advances: improved Yau–Yau and YauYauAL software

# Signal–observation model (diffusion case)

We consider a continuous-time hidden Markov model:

$$
\begin{aligned}
d\boldsymbol{X}_t &= \boldsymbol{f}(\boldsymbol{X}_t, t)\,dt + G(\boldsymbol{X}_t, t)\,d\boldsymbol{W}_t, \qquad \boldsymbol{X}_t \in \mathbb{R}^n, \\
d\boldsymbol{Y}_t &= \boldsymbol{h}(\boldsymbol{X}_t, t)\,dt + H(\boldsymbol{X}_t, t)\,d\boldsymbol{V}_t, \qquad \boldsymbol{Y}_t \in \mathbb{R}^m,
\end{aligned}
$$

where $\boldsymbol{W}_t \in \mathbb{R}^r$, $\boldsymbol{V}_t \in \mathbb{R}^m$ are independent standard Brownian motions.

**Filtering problem.** Given the observation history $\mathcal{Y}_t := \sigma(\boldsymbol{Y}_s : 0 \le s \le t)$, estimate $\boldsymbol{X}_t$ (or functionals of $\boldsymbol{X}_t$) in real time.

# What does "optimal filtering" mean?

For a test function $\varphi : \mathbb{R}^n \to \mathbb{R}$ (bounded or with suitable growth), the optimal (MMSE) estimator is the conditional expectation

$$\pi_t(\varphi) := \mathbb{E}\big[\varphi(\boldsymbol{X}_t) \mid \mathcal{Y}_t\big].$$

Equivalently, $\pi_t$ is the *conditional law* of $\boldsymbol{X}_t$ given $\mathcal{Y}_t$.

If $\pi_t$ admits a density $\rho(t, x)$ w.r.t. Lebesgue measure,

$$\pi_t(\varphi) = \int_{\mathbb{R}^n} \varphi(x)\rho(t, x)\, dx, \qquad \int_{\mathbb{R}^n} \rho(t, x)\, dx = 1.$$

**Goal:** compute $\rho(t, \cdot)$ (or moments) sequentially.

# Generator of the signal (forward operator)

Let $\mathcal{L}$ be the (time-dependent) generator of the signal diffusion:

$$(\mathcal{L}_t\varphi)(x) = \sum_{i=1}^{n} f_i(x,t)\,\partial_{x_i}\varphi(x) + \frac{1}{2}\sum_{i,j=1}^{n} a_{ij}(x,t)\,\partial^2_{x_i x_j}\varphi(x), \quad a := GG^\top.$$

The adjoint operator $\mathcal{L}_t^*$ appears in the Fokker–Planck (Kolmogorov forward) equation for the *unconditional* density $p(t,x)$:

$$\partial_t p = \mathcal{L}_t^* p.$$

In filtering, the density evolves *conditioned* on observations, and becomes a (stochastic) PDE.

# Two key ideas behind the Zakai/DMZ equation

1. **Unnormalized density.** Instead of the normalized conditional density $\rho$, introduce an *unnormalized* conditional density $\sigma$ so that normalization becomes a final step:

$$\rho(t,x) = \frac{\sigma(t,x)}{\int_{\mathbb{R}^n} \sigma(t,z)\,dz}.$$

2. **Change of measure (Girsanov).** Under a new probability measure, the observation process becomes a Brownian motion. This "linearizes" the filtering equation (at the cost of working with $\sigma$).

# Girsanov change of measure (whitening the observations)

For simplicity assume $H \equiv I_m$ (observation noise is standard). Define the likelihood (Radon–Nikodym) process

$$\Lambda_t := \exp\Big( -\int_0^t \langle \boldsymbol{h}(\boldsymbol{X}_s, s), d\boldsymbol{Y}_s \rangle + \frac{1}{2}\int_0^t \|\boldsymbol{h}(\boldsymbol{X}_s, s)\|^2 \, ds \Big).$$

Assuming Novikov's condition, define a new measure $\mathbb{Q}$ on $\mathcal{F}_t$ by

$$\frac{d\mathbb{Q}}{d\mathbb{P}}\Big|_{\mathcal{F}_t} = \Lambda_t.$$

Then (under $\mathbb{Q}$) the process $\boldsymbol{Y}_t$ is an $m$-dimensional Brownian motion.

This is the standard route to the **Kallianpur–Striebel formula**.

# Kallianpur–Striebel formula

For suitable $\varphi$,

$$\pi_t(\varphi) = \frac{\mathbb{E}_{\mathbb{Q}}\left[\varphi(\boldsymbol{X}_t)\Lambda_t^{-1} \mid \mathcal{Y}_t\right]}{\mathbb{E}_{\mathbb{Q}}\left[\Lambda_t^{-1} \mid \mathcal{Y}_t\right]}.$$

Define the **unnormalized filter** (a random measure on $\mathbb{R}^n$)

$$\Sigma_t(\varphi) := \mathbb{E}_{\mathbb{Q}}\left[\varphi(\boldsymbol{X}_t)\Lambda_t^{-1} \mid \mathcal{Y}_t\right].$$

Then

$$\pi_t(\varphi) = \frac{\Sigma_t(\varphi)}{\Sigma_t(1)}.$$

If $\Sigma_t$ has density $\sigma(t,x)$, then $\Sigma_t(\varphi) = \int \varphi(x)\sigma(t,x)\,dx$.

# Zakai/DMZ equation in weak form

Let $\Sigma_t(\varphi)$ be as defined. A standard Itô calculation yields the **Duncan–Mortensen–Zakai (DMZ) equation**:

$$d\Sigma_t(\varphi) = \Sigma_t(\mathcal{L}_t\varphi)\,dt + \sum_{k=1}^{m} \Sigma_t\big(\varphi\, h_k(\cdot, t)\big)\,dY_t^k,$$

where $(h_1, \ldots, h_m) = \boldsymbol{h}$.

**Key point:** this is *linear* in $\Sigma_t$ (contrast with the normalized Kushner–Stratonovich equation).

# Density form: linear SPDE (Zakai equation)

Assume $\Sigma_t$ admits a density $\sigma(t,x)$. Then $\sigma$ solves

$$d\sigma(t,x) = (\mathcal{L}_t^* \sigma)(t,x)\, dt + \sum_{k=1}^{m} h_k(x,t)\, \sigma(t,x)\, dY_t^k.$$

After solving for $\sigma$, normalize:

$$\rho(t,x) = \frac{\sigma(t,x)}{\int_{\mathbb{R}^n} \sigma(t,z)\, dz}.$$

This is the fundamental *PDE/SPDE* viewpoint of nonlinear filtering.

# Connection to Kushner–Stratonovich (normalized) equation

Define $\pi_t(\varphi) = \Sigma_t(\varphi)/\Sigma_t(1)$ and innovations

$$dI_t^k := dY_t^k - \pi_t(h_k(\cdot, t))\, dt.$$

Then the normalized filter satisfies

$$d\pi_t(\varphi) = \pi_t(\mathcal{L}_t\varphi)\, dt + \sum_{k=1}^{m} \Big(\pi_t(\varphi h_k) - \pi_t(\varphi)\pi_t(h_k)\Big)\, dI_t^k,$$

which is *nonlinear*.

In practice, the linearity of Zakai/DMZ is advantageous for analysis and numerics.

# Specialization used in classical Yau–Yau papers

A common setting (including the attached Yau–Yau paper) assumes:

$$dX_t = f(X_t)\,dt + dW_t, \qquad dY_t = h(X_t)\,dt + dV_t,$$

i.e. identity diffusion and identity observation noise.

In this case the DMZ SPDE for the unnormalized density can be written as

$$d\sigma(t,x) = L_0\sigma(t,x)\,dt + \sum_{i=1}^{m} L_i\sigma(t,x)\,dY_t^i,$$

with $L_i$ multiplication by $h_i$ and

$$L_0 = \frac{1}{2}\Delta - \sum_{j=1}^{n} f_j\,\partial_{x_j} - \sum_{j=1}^{n}(\partial_{x_j} f_j) - \frac{1}{2}\sum_{i=1}^{m} h_i^2.$$

# Why a "robust" (pathwise) formulation?

The DMZ/Zakai equation is a stochastic PDE driven by $d\boldsymbol{Y}_t$. In many applications we have a *single observed sample path* $t \mapsto \boldsymbol{Y}_t(\omega)$, and we want algorithms stable under small perturbations of the observed path.

A widely-used approach is to transform DMZ into a *random PDE*: the observation path enters as a time-dependent coefficient (no stochastic differential).

This is the starting point for the Yau–Yau "without memory" filtering strategy.

# Exponential transform and robust DMZ

Define the transformed (unnormalized) density

$$u(t,x) := \exp\Big( -\sum_{i=1}^{m} h_i(x)\, Y_t^i \Big)\, \sigma(t,x).$$

Then $u$ satisfies a **time-varying parabolic PDE**

$$\partial_t u(t,x) = L_0 u(t,x) + \sum_{i=1}^{m} Y_t^i\, [L_0, L_i] u(t,x) + \frac{1}{2} \sum_{i,j=1}^{m} Y_t^i Y_t^j\, [[L_0, L_i], L_j] u(t,x),$$

where $[\cdot,\cdot]$ is the Lie bracket of differential operators.

# Expanded robust DMZ PDE (time-varying coefficients)

In the same setting ($dX = f(X)dt + dW$, $dY = h(X)dt + dV$), the robust DMZ can be written explicitly as

$$\begin{aligned}\partial_t u = \frac{1}{2}\Delta u - \sum_{j=1}^{n} f_j(x)\,\partial_{x_j} u - \sum_{j=1}^{n}(\partial_{x_j} f_j(x))\,u - \frac{1}{2}\sum_{i=1}^{m} h_i(x)^2\,u \\ + \sum_{i=1}^{m}\sum_{j=1}^{n} Y_t^i\,(\partial_{x_j} h_i(x))\,\partial_{x_j} u + \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n} Y_t^i\,\partial^2_{x_j x_j} h_i(x)\,u \\ - \sum_{i=1}^{m}\sum_{j=1}^{n} Y_t^i\,f_j(x)\,(\partial_{x_j} h_i(x))\,u + \frac{1}{2}\sum_{i,j=1}^{m}\sum_{k=1}^{n} Y_t^i Y_t^j\,(\partial_{x_k} h_i)(\partial_{x_k} h_j)\,u.\end{aligned}$$

# Core challenge: real-time & without memory

The robust DMZ PDE depends on the whole observation path $t \mapsto \boldsymbol{Y}_t$ through time-varying coefficients. A naive real-time solver would require storing the entire observation history.

**Goal (Yau–Yau):**

- compute the filter in *real time*;
- require only the observation values at the *current* (and previous) time;
- move heavy computation *offline*.

# Freezing the observation on short intervals

Let $0 = \tau_0 < \tau_1 < \cdots < \tau_k = \tau$ be a partition of $[0, \tau]$. On each interval $[\tau_{i-1}, \tau_i]$, approximate the observation path by the constant value $\boldsymbol{Y}_{\tau_i}$:

$$\boldsymbol{Y}_t \approx \boldsymbol{Y}_{\tau_i}, \qquad t \in [\tau_{i-1}, \tau_i].$$

This converts the time-varying robust DMZ PDE into a PDE with *constant-in-time coefficients* over each step.

Yau–Yau show that as $|P_k| := \max_i(\tau_i - \tau_{i-1}) \to 0$, the resulting piecewise scheme converges to the true robust DMZ solution.

# Key transform (Proposition 3.1 idea)

Fix an interval $[\tau_{i-1}, \tau_i]$ and freeze $\boldsymbol{Y}_t \equiv \boldsymbol{Y}_{\tau_i}$. Let $u(t,x)$ solve the frozen robust DMZ on this interval.

Define

$$u(t,x) = \exp\Big( -\sum_{\ell=1}^{m} Y_{\tau_i}^{\ell} h_\ell(x) \Big)\, \tilde{u}(t,x).$$

Then $\tilde{u}$ satisfies a **parabolic PDE independent of observations** (on this interval).

This is the backbone of the offline-online decomposition.

# Observation-independent PDE on each step

Under the assumptions of the classical Yau–Yau setting, $\tilde{u}$ satisfies

$$\partial_t \tilde{u} = \frac{1}{2}\Delta \tilde{u} - \sum_{j=1}^{n} f_j(x)\, \partial_{x_j} \tilde{u} - \Big( \sum_{j=1}^{n} \partial_{x_j} f_j(x) + \frac{1}{2} \sum_{\ell=1}^{m} h_\ell(x)^2 \Big) \tilde{u}, \qquad t \in [\tau_{i-1}, \tau_i].$$

**Crucial:** no $\boldsymbol{Y}_t$ appears in the PDE operator (only in the exponential prefactor / initial condition).

# Recursive scheme on a partition

Let $u(\cdot,\cdot)$ denote the (exact) robust DMZ solution, and construct approximations $u_i$ on each interval $[\tau_{i-1},\tau_i]$ by freezing observations.

Yau–Yau prove that

$$u(\tau,x) = \lim_{|P_k|\to 0} u_k(\tau_k,x),$$

where $u_k(\tau_k,\cdot)$ is obtained by iterating the stepwise frozen PDEs.

# Initialization ($i = 1$)

On $[0, \tau_1]$ solve the observation-independent PDE for $\tilde{u}_1$:

$$\partial_t \tilde{u}_1 = \frac{1}{2}\Delta \tilde{u}_1 - \sum_{j=1}^{n} f_j\, \partial_{x_j} \tilde{u}_1 - \Big( \sum_{j=1}^{n} \partial_{x_j} f_j + \frac{1}{2} \sum_{\ell=1}^{m} h_\ell^2 \Big) \tilde{u}_1, \quad \tilde{u}_1(0, x) = \sigma_0(x) \exp\Big( \sum_{\ell=1}^{m} Y_{\tau_1}^{\ell} h_\ell(x) \Big).$$

Then recover the frozen-step approximation at time $\tau_1$ by

$$u_1(\tau_1, x) = \exp\Big( - \sum_{\ell=1}^{m} Y_{\tau_1}^{\ell} h_\ell(x) \Big)\, \tilde{u}_1(\tau_1, x).$$

# General step ($i \geq 2$): online update enters the IC

Given $\tilde{u}_{i-1}(\tau_{i-1}, \cdot)$, define the next initial condition by multiplying only with the *increment* of observations:

$$\tilde{u}_i(\tau_{i-1}, x) = \exp\Big( \sum_{\ell=1}^{m} \big( Y^{\ell}_{\tau_i} - Y^{\ell}_{\tau_{i-1}} \big) \, h_\ell(x) \Big) \, \tilde{u}_{i-1}(\tau_{i-1}, x).$$

Then evolve $\tilde{u}_i$ from $\tau_{i-1}$ to $\tau_i$ with the *same* observation-independent PDE operator. Finally,

$$u_i(\tau_i, x) = \exp\Big( - \sum_{\ell=1}^{m} Y^{\ell}_{\tau_i} h_\ell(x) \Big) \, \tilde{u}_i(\tau_i, x).$$

# Offline-online separation (what is computed when?)

**Offline (once):**

- choose time step $\Delta t$ and spatial domain / grid;
- precompute the action of the semigroup generated by the observation-independent operator

$$\mathcal{A} := \frac{1}{2}\Delta - \sum_{j=1}^{n} f_j \partial_{x_j} - \Big( \sum_{j=1}^{n} \partial_{x_j} f_j + \frac{1}{2} \sum_{\ell=1}^{m} h_\ell^2 \Big),$$

  e.g. via finite differences, spectral methods, or precomputed Green's functions when available.

**Online (each step):**

- multiply by $\exp\big((\boldsymbol{Y}_{\tau_i} - \boldsymbol{Y}_{\tau_{i-1}}) \cdot \boldsymbol{h}(x)\big)$ (cheap);
- apply one offline propagator step (matrix-vector multiply / PDE solve);
- normalize to obtain $\rho$.

# Why "without memory"?

At step $i$ the algorithm needs only:

$$\boldsymbol{Y}_{\tau_{i-1}}, \quad \boldsymbol{Y}_{\tau_i}.$$

All earlier observation values are irrelevant because they have already been absorbed into $\tilde{u}_{i-1}(\tau_{i-1}, \cdot)$.

This is a major practical advantage over methods that require full access to the observation path.

**Interpretation:** the filter state (a density function / its discretization) is the sufficient statistic.

# Numerical realization choices

The Yau–Yau framework is compatible with many PDE solvers for the observation-independent evolution:

- Finite differences (implicit / semi-implicit) for stability;
- Spectral / Hermite methods for smooth problems;
- Kernel / Green's function methods if the fundamental solution is explicit ("Yau systems");
- Operator-theoretic compression / low-rank approximations for higher dimensions.

The online multiplication by $\exp((\Delta \boldsymbol{Y}) \cdot \boldsymbol{h}(x))$ is pointwise and trivially parallelizable.

# Accuracy, convergence, and step size

Sources of error typically include:

- time discretization (freezing $\boldsymbol{Y}_t$ on steps);
- numerical error from the PDE solver (grid, truncation, stability);
- domain truncation / boundary conditions in high dimensions.

Yau–Yau provide pointwise and $L^2$ convergence results for the freezing approximation as $|P_k| \to 0$, under suitable regularity and growth conditions on $f, h$ and the observation path.

# High-dimensional nonlinear filtering: the pain points

Classical methods struggle as dimension grows:

- Particle filters: exponential particle explosion; weight degeneracy.
- EKF/UKF: rely on (near-)Gaussian posteriors; can fail for multimodal or highly nonlinear sensors.
- Grid-based Zakai solvers: curse of dimensionality ($N^n$ grid points).

Modern Yau–Yau variants aim to preserve:

$$\textit{offline heavy lift} + \textit{online lightweight updates}$$

while scaling to very high $n$.

# Improved Yau–Yau (2025): key innovations

The improved framework integrates:

- low-discrepancy (QMC) sampling for state-space coverage;
- a novel offline-online update with error bounds;
- high-order multi-scale kernel approximation for the Kolmogorov propagator;
- fully log-domain likelihood computation for numerical stability;
- local resampling–restart to prevent weight collapse;
- CPU/GPU parallel acceleration (offline kernel assembly + online matrix-vector multiplies).

# Improved offline/online split (conceptual)

Consider the (unnormalized) DMZ evolution over one step of size $\Delta t$.

**Offline prediction (pure dynamics):** propagate by the Kolmogorov forward equation

$$\partial_t p = \mathcal{L}^* p,$$

independent of the data.

**Online correction (likelihood):** upon receiving $\Delta \boldsymbol{Y}_k := \boldsymbol{Y}_{t_{k+1}} - \boldsymbol{Y}_{t_k}$, apply a multiplicative update

$$p_{k+1}(x) \;\propto\; p_{k+1|k}(x) \exp\left(\boldsymbol{h}(x)^\top \Delta \boldsymbol{Y}_k - \frac{1}{2}\|\boldsymbol{h}(x)\|^2 \Delta t\right),$$

including the quadratic Itô correction term.

# Procedure summary (improved Yau–Yau algorithm)

1. Choose domain $\Omega$ and generate sample points $\{x^{(j)}\}_{j=1}^{N}$ using low-discrepancy sequences (Halton/Sobol).
2. **Offline:** build a discrete propagator for the Kolmogorov forward equation (e.g. as an $N \times N$ matrix mapping weights at $t_k$ to $t_{k+1}$).
3. **Online:** for each observation increment $\Delta \boldsymbol{Y}_k$, update weights by a log-likelihood factor (linear + quadratic terms).
4. Normalize and compute estimates by Monte Carlo quadrature:

$$\hat{x}_k = \sum_{j=1}^{N} w_k^{(j)} x^{(j)}.$$

# Provable accuracy: local and global bounds (headline)

The improved scheme provides theoretical truncation-error and stability guarantees.

A representative result (informal): if $\Delta t$ is the time step and $D^*(N)$ is the star-discrepancy of the QMC point set, then

$$\text{local error} \sim O(\Delta t^2 + D^*(N)), \qquad \text{global error} \sim O(\Delta t + D^*(N)/\Delta t).$$

These bounds motivate:

- small $\Delta t$ (time accuracy);
- low-discrepancy sampling (small $D^*(N)$) to tame dimension.

# High-order multi-scale kernel approximation (idea)

Instead of gridding $\Omega \subset \mathbb{R}^n$, approximate the propagator

$$p(t_{k+1}, \cdot) \approx e^{\Delta t\, \mathcal{L}^*}\, p(t_k, \cdot)$$

by a kernel expansion evaluated on QMC points:

$$p_{k+1}(x^{(i)}) \approx \sum_{j=1}^{N} K_{\Delta t}(x^{(i)}, x^{(j)})\, p_k(x^{(j)}).$$

Multi-scale / higher-order kernels improve accuracy while remaining amenable to parallel computation.

# Log-domain computation: preventing underflow/overflow

In high dimension, products of small likelihood factors can underflow. Work with log-weights:

$$\ell_k^{(j)} := \log w_k^{(j)}.$$

Prediction and update use **log-sum-exp** stabilization:

$$\log\Big(\sum_j e^{a_j}\Big) = a_{\max} + \log\Big(\sum_j e^{a_j - a_{\max}}\Big).$$

Online update (schematic):

$$\ell_{k+1}^{(j)} \leftarrow \ell_{k+1|k}^{(j)} + \boldsymbol{h}(x^{(j)})^\top \Delta \boldsymbol{Y}_k - \tfrac{1}{2}\|\boldsymbol{h}(x^{(j)})\|^2 \Delta t.$$

# Local resampling–restart mechanism (intuition)

Even with QMC points, weight collapse can occur in strongly nonlinear regimes.

A local resampling–restart strategy:

- detect weight degeneracy (e.g. low effective sample size);
- resample points locally around high-probability regions;
- restart the filter with refreshed local point clouds.

Goal: retain multi-modality and avoid collapsing onto a single mode.

# Parallel acceleration: CPU/GPU split

The improved algorithm is designed for parallelism:

- **Offline (CPU multi-threading):** assemble kernel/propagator matrix rows in parallel.
- **Online (GPU):** dominant cost is repeated matrix-vector multiplies $p_{k+1|k} = Kp_k$.
- **Online (CPU):** pointwise likelihood update and normalization are embarrassingly parallel.

This makes real-time filtering feasible for $n$ in the hundreds to thousands, given sufficient hardware.

# Empirical performance (headline)

Reported experiments include:

- large-scale cubic sensor tests up to $n \approx 1000$;
- small-scale highly nonlinear tests comparing EKF/UKF/PF;
- linear-Gaussian benchmarks comparing to Kalman–Bucy.

Typical qualitative outcomes:

- sub-quadratic runtime scaling in $n$ with parallelization;
- sub-linear growth of estimation error versus dimension;
- strong robustness in multimodal / strongly nonlinear regimes.

# YauYauAL software: what it provides

**YauYauAL** is an R package + Shiny app to make Yau–Yau filtering easier to use:

- R interface with computational kernels in C++ via `Rcpp`;
- interactive GUI for parameter tuning and visualization;
- finite-difference solvers for Kolmogorov forward equations;
- modular structure to plug in different models and numerical schemes.

# YauYauAL: installation and quick start (R)

## Install and run the Shiny interface

```
install.packages("devtools")
devtools::install_github("BIMSA-Stat/YauYauAL")

library(YauYauAL)
YauYauAL::run_app()
```

The package includes step-by-step templates (e.g. cubic sensor examples) for defining $f$, $h$, time discretization, and solver settings.

# Workflow checklist for practitioners

1. **Modeling:** specify $f$, $G$ and $h$, $H$; verify regularity/growth assumptions.
2. **Discretization:** choose $\Delta t$, domain truncation, boundary conditions.
3. **Offline stage:** precompute propagator (PDE solve / kernel matrix).
4. **Online stage:** stream observations; update by exponential likelihood; normalize.
5. **Diagnostics:** monitor effective sample size / stability; trigger resampling if needed.

# Summary and outlook

- DMZ/Zakai converts nonlinear filtering into a *linear* SPDE for an unnormalized density.
- The Yau–Yau strategy transforms the robust DMZ into an *offline-online* scheme: observation-independent PDE propagation + cheap multiplicative updates.
- Recent work improves scalability with QMC sampling, kernel approximations, log-domain updates, and parallelization.
- Software (YauYauAL) aims to lower the barrier to applying Yau–Yau filters in practice.

# Selected references


M. Zakai.
On the optimal filtering of diffusion processes.
*Z. Wahrscheinlichkeitstheorie verw. Gebiete*, 11(3):230–243, 1969.

S.-T. Yau and S. S.-T. Yau.
Real time solution of nonlinear filtering problem without memory I.
*Mathematical Research Letters*, 7:671–693, 2000.

S.-T. Yau and Y.-S. Niu.
An Improved Yau–Yau Algorithm for High Dimensional Nonlinear Filtering Problems.
*arXiv:2509.16896*, 2025.

Y. Wang, S. Xu, X. Wei, X. Luo, S. S.-T. Yau, S.-T. Yau, R. Wu.
Yau-YauAL: A computer tool for solving nonlinear filtering problems.
*arXiv:2506.08976*, 2025.